Séquence 1

COURS - Conception & Mise au point de programmes

Repère historique

John McCarthy *(1927-2011)* auteur du langage Lisp en 1958 qui est le deuxième langage ancien encore largement diffusé aujourd'hui. Il joua un rôle majeur dans la programmation en intelligence artificielle, écrivant un des premiers programmes jouant aux échecs.

Qu'est-ce qu'une fonction ?

Les programmeurs décomposent souvent leur programmes en *blocs* courts. Chaque bloc de code, associé à un nom, est appelé **fonction**.
Procéder ainsi, permet:

- d'éviter les répétitions et donc de *factoriser* le code;
- de faciliter la maintenance;
- de faciliter dans une certaine mesure le travail en équipe.

> **Note**
>
> D'autres termes comme sous-programme, procédure, etc. sont parfois employés, à quelques nuances près. On ne les utilisera pas en spécialité NSI.

Comment définir et appeler une fonction ?

La syntaxe présentée ici est celle du langage Python. On définit une fonction avec:

- le mot clé `def` suivi d'un nom;
- une paire de parenthèses à l'intérieur desquelles figurent éventuellement des paramètres;
- le caractère *deux-points* `:` qui termine la ligne de déclaration;
- un bloc de code indenté qui forme le corps de la fonction;
- le mot clé `return` (optionnel)

```python
def nom_fonction(paramètre(s)):
    bloc_instructions
```

Le nom de la fonction **doit** commencer par une lettre et **ne doit pas** être un mot reservé. Ce nom sera le plus explicite possible. La fonction peut avoir 0, 1 ou plus de paramètres.

Exemple

```python
def deux_puissance(n):
    p = 1
    for c in range(n):
        p = p * 2
    return p
```

Lorsqu'on exécute ce code, il ne se passe **rien**. A ce stade on a définit la fonction, il faut maintenant l'appeler pour que tout son code soit exécuté. L'appel d'une fonction consiste à écrire **son nom suivi de parenthèses ouvrantes-fermantes** à l'intérieur desquelles on place d'éventuels **arguments**.

```python
# Calcul et affichage de 2^8 et 2^16
print(deux_puissance(8))
print(deux_puissance(16))
```

```
256
65536
```

Note

- En l'absence d'instruction `return`, python renvoie *implicitement* une valeur particulière `None` qui signifie 'rien';
- une fonction peut appeler une autre fonction (concept développé en classe de terminale).

Spécifier une fonction

Qu'est-ce qu'une spécification ?

En plus d'un nom explicite, une fonction devrait être convenablement documentée. Cette documentation devra comporter les **spécifications** de la fonction.

Spécifier une fonction consiste à donner:

- la liste ordonnée et le type de ses paramètres;
- une description du résultat renvoyé ou **post-conditions**;

- les conditions ou propriétés portant sur les paramètres d'entrée ou **pré-conditions**.

En python, la documentation suit **immédiatement** la définition de la fonction et est encadrée par des triples double quotes `"""`.

Pour garantir les post-conditions ou les pré-conditions, on peut utiliser des **assertions**.

> **Definition**
>
> Une assertion est une proposition logique qui doit être vraie (`True` en python) sinon le programme est arrêté.

La syntaxe d'une assertion en python est:

```
assert expression, "Message d'erreur explicite"
```

Par ailleurs, la documentation d'une fonction est en outre accessible avec la fonction native `help()`

Exemple complet

Spécifions la fonction `deux_puissance` précédente. On utilisera la fonction native de python `isinstance` pour vérifier que le type d'un objet est bien celui attendu.

```
def deux_puissance(n):
    """
    renvoie la puissance n-ième de 2;
    n: entier naturel.
    """
    # Préconditions
    assert isinstance(n, int), "Erreur: n entier"
    assert n >= 0, "Erreur: n >= 0"

    p = 1
    for c in range(n):
        p = p * 2
    return p
```

```
deux_puissance(20)
```

```
1048576
```

En cas de non respect des préconditions, le *contrat* entre le programmeur et l'utilisateur est rompu et le programme s'arrête. Par exemple, un appel `deux_puissance(2.5)` conduit à:

```
---------------------------------------------------------------------------

AssertionError                            Traceback (most recent call last)

<ipython-input-11-34ff1cbea852> in <module>
      1 # Non respect de: "n, entier naturel".
      2
----> 3 deux_puissance(2.5)


<ipython-input-9-5988c4a4a8b7> in deux_puissance(n)
      5     """
      6     # Préconditions
----> 7     assert isinstance(n, int), "Erreur: n entier"
      8     assert n >= 0, "Erreur: n >= 0"
      9


AssertionError: Erreur: n entier
```

L'accès à la documentation se fait avec `help(deux_puissance)`.

```
Help on function deux_puissance in module __main__:

deux_puissance(n)
    renvoie la puissance n-ième de 2;
    n: entier naturel.
```

Utiliser des bibliothèques

La plupart des langages de programmation proposent des fonctions toutes prêtes. Ces fonctions fournies avec le langage sont regroupées au sein d'une **bibliothèque standard**. Les fonctions sont regroupées par *modules* spécialisés (par exemple *math* ou *random*).

Pour pouvoir utiliser les objets (*des fonctions par exemple*) d'un module, il faut les **importer** avec l'instruction `import`. On privilégiera la notation `from module import objet` au lieu de `from module import *`.

Exemple

```python
# Utilisation de la fonction racine carrée (sqrt) du module math
from math import sqrt

print(sqrt(2))
```

```
1.4142135623730951
```

Une autre construction possible et aussi valable que la précédente, consiste à importer le module avec `import module` puis à utiliser l'objet avec la notation *pointée* `module.objet`.

Exemple

```python
import math

print(math.sqrt(2))
```

```
1.4142135623730951
```

Tester son code

Pour s'assurer que le programme fonctionne comme prévu, il est nécessaire de le tester. Un jeu de tests devrait couvrir les cas d'utilisation:

- typiques;
- limites.

On peut utiliser des assertions dans un premier temps (l'utilisation d'un module de test spécialisé peut être envisagé en python).

Enfin, une approche moderne consiste à écrire ses tests avant les fonctions.

Exemple

```python
# Jeu de tests

def test_puissance():
    # Cas limite
    assert deux_puissance(0) == 1
    # Cas typique
    assert deux_puissance(10) == 1024
    return 'Tests OK'

test_puissance()
```

```
'Tests OK'
```

IMPORTANT

Le succès d'un jeu de test ne garantit pas que le programme ne contient pas de bugs. Ce succès est une condition nécessaire mais non suffisante.